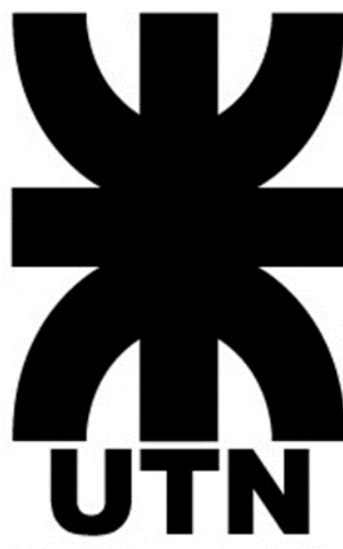




Universidad Tecnológica Nacional
Facultad Regional
Concepción del Uruguay
Ingeniería en Sistemas de Información

Bases de Datos

2026

TRABAJO PRÁCTICO INTEGRADOR

"BibliolA: Sistema de Gestión de Biblioteca
con Agente de Inteligencia Artificial"

Alumnos: Buet Mia, Díaz Emmanuel, Farabello Luciana, Rodriguez Rodrigo y Sigales Maria Juliana.

Docentes: Ing. Francisco Fazzio, Ing. Pablo Colombo.

Índice

Introducción.....	3
Contexto.....	3
Objetivos.....	3
➤ Objetivo General.....	3
➤ Objetivos Específicos.....	4
Modelo Relacional.....	4
Diagrama de clases.....	4
Decisiones de diseño.....	5
Stored procedures y triggers.....	7
Stored procedures.....	7
Triggers.....	8
Arquitectura del agente IA.....	10
Prompt de Sistema.....	10
Evaluación del agente.....	14
Módulo de recomendaciones.....	20
Conclusiones.....	22
Bibliografía y recursos utilizados.....	23

Trabajo Práctico Integrador

"BibliolA: Sistema de Gestión de Biblioteca con Agente de Inteligencia Artificial"

Introducción

El acceso a la información almacenada en sistemas de gestión de bases de datos relacionales ha estado históricamente supeditado al uso de lenguajes de consulta estructurados o al desarrollo de interfaces de usuario predefinidas. Esto genera una barrera para los usuarios que no tienen conocimientos técnicos y necesitan respuestas rápidas para tomar decisiones en el día a día.

El paradigma de la Inteligencia Artificial Generativa y los Modelos de Lenguaje de Gran Escala (LLM) ofrece una alternativa para resolver este problema mediante interfaces que entienden el lenguaje natural. Sin embargo, dejar que una IA maneje directamente las reglas de un negocio o la carga de datos es peligroso, esto hace que se introduzcan riesgos de inconsistencia y alucinaciones.

Bajo esta premisa, se lleva adelante el Trabajo Práctico Integrador con la formulación de **BibliolA**, un sistema de gestión de biblioteca.

Contexto

El proyecto se desarrolla para resolver las necesidades operativas de una biblioteca, un entorno que maneja un flujo constante de préstamos, devoluciones y control de stock de libros. La complejidad de este negocio está en la cantidad de reglas que se deben cumplir estrictamente como: verificar que queden libros disponibles antes de prestarlos, controlar que un socio no supere el límite de retiros permitidos y aplicar sanciones automáticas si alguien devuelve un libro fuera de término.

Para que el sistema sea seguro, no se puede dejar que la IA lea y escriba directamente sobre las tablas principales de la base de datos. Por eso, el contexto de este trabajo plantea una división de tareas claras: las reglas de negocio, el control de las transacciones y las restricciones de seguridad se programan por completo dentro del motor de base de datos. De esta manera, el agente de IA solo actúa como un traductor que facilita las preguntas del usuario, garantizando que el núcleo del sistema se mantenga intacto y protegido contra cualquier error del modelo.

Objetivos

> Objetivo General

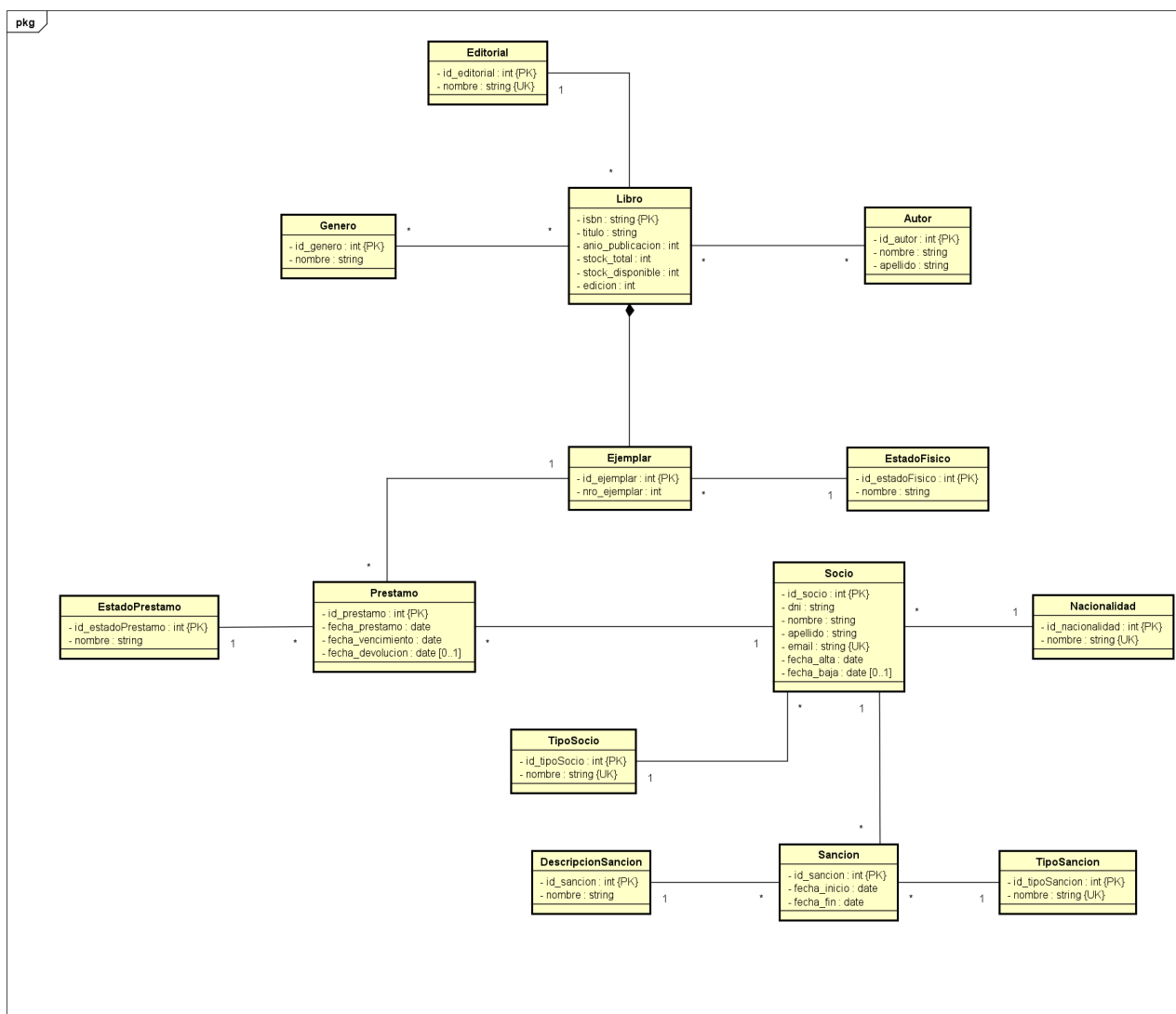
- Diseñar e implementar un sistema de gestión de biblioteca que combine una base de datos relacional con un agente de inteligencia artificial, logrando que el usuario pueda hacer consultas en lenguaje natural mientras el motor de base de datos se encarga de asegurar el cumplimiento de las reglas y la protección de los datos.

➤ Objetivos Específicos

- Diseñar una estructura de datos ordenada. Organizar tablas principales del sistema utilizando reglas de normalización para evitar que la información se duplique o se cargue con inconsistencias.
- Asegurar las reglas de negocio desde el motor.
- Simplificar la información que lee la IA, reduciendo la posibilidad de la confusión de columnas y la generación de consultas equivocadas por parte del modelo.
- Desarrollar un sistema que reciba la pregunta del usuario en español, la acople con las instrucciones de la base de datos para que la IA genere el código SQL correcto, y muestre los resultados en pantalla de forma transparente.

Modelo Relacional

Diagrama de clases



Decisiones de diseño

Políticas de Tipos de Datos

Para todas las claves primarias se utilizó el tipo *int* (integer) combinado con la propiedad *auto_increment*. Esto asegura un indexado veloz, evitando manejar claves complejas creadas de forma manual. La única excepción es el *isbn* de la tabla *libro*, donde se respetó su naturaleza de texto por ser un estándar internacional.

Se implementó *varchar* con longitudes acotadas según el tipo de dato. Al utilizar *varchar* en lugar de *char*, el motor solo consume el espacio real del texto introducido, optimizando el uso de la memoria en disco.

Para los atributos que registran eventos operativos (*fecha_prestamo*, *fecha_vencimiento*, *fecha_devolucion*, *fecha_alta*, *fecha_inicio*, *fecha_fin*) se utilizó el tipo *date*, ya que solo se requiere precisión a nivel de día. En cambio, para la tabla *auditoria_prestamo* se configuró *timestamp default current_timestamp*, lo que permite registrar de forma automática el tiempo exacto en el que ocurrió cualquier modificación.

Para evitar la duplicación de datos estratégicos y asegurar la identidad de ciertas entidades, se definieron restricciones de clave única (*unique key*) en columnas específicas:

- *email* en tabla *socio*: se definió como clave única al correo electrónico de un socio para asegurarse de que cada uno de ellos tenga una dirección distinta asignada en el sistema impidiendo registros duplicados. Se ajustó con un tamaño de 150 caracteres para permitir correos extensos
- *nombre* en tabla *editorial*: se configuró de esta forma para evitar que se cargue la misma empresa editorial más de una vez.
- *nombre* en tabla *nacionalidad*: al igual que con las editoriales, se restringió este campo para garantizar que no existan filas duplicadas para un mismo país.
- *nombre* en tabla *género*: se aplicó la restricción de unicidad para asegurar que cada categoría literaria exista una sola vez, manteniendo limpio el catálogo y consistente para los filtros del sistema y las búsquedas de la IA.

No se definió el campo DNI como clave única en la tabla *socio*, contemplando la existencia de personas con números de documento duplicados.

Políticas de Índices

- **idx_dni_socio**

Justificación: al indexar el DNI se permite que el motor localice un socio de forma casi instantánea en registros de préstamos y sanciones. En las búsquedas del asistente, el DNI es el filtro de coincidencia exacta más usado para identificar a un usuario. Internamente, esto permite al motor realizar búsquedas directas en el árbol secundario y obtener la clave primaria (*id_socio*) mediante un salto directo de memoria, evitando escanear la tabla de socios fila por fila.

- **idx_apellido_nombre_socio**

Justificación: cuando los bibliotecarios buscan socios por sus datos filiatorios o cuando se requiere presentar reportes ordenados alfabéticamente, este índice acelera drásticamente

las operaciones de ordenamiento y las búsquedas. Su uso es crítico para el rendimiento de las vistas *vista_socio_activos*, *vista_prestamos_activos*, *vista_prestamos_vencidos* y *vistas_sanciones_activas*, ya que todas ellas realizan la función de concatenación para mostrar al usuario final el nombre completo ordenado.

- **idx_titulo_libro**

Justificación: el título de los libros es el campo de búsqueda más común e informal que realizan los usuarios. Esto impacta reduciendo el tiempo de lectura en disco cuando se buscan obras específicas.

- **idx_genero**

Justificación: el género literario es un filtro de agrupación recurrente en las búsquedas de una biblioteca. Este índice es fundamental para acelerar los cruces de datos en la *vista_catalogo_libros*, la cual ejecuta funciones de agregación de texto (GROUP_CONCAT) para listar las categorías de cada obra. Al indexar el nombre del género, el motor procesa las uniones con la tabla intermedia *generoLibro* de forma mucho más veloz al encontrarse los términos ordenados en memoria.

- **idx_apellido_nombre_autor**

Justificación: sucede igual que con los géneros, las búsquedas del material de un autor son recurrentes en una biblioteca. Agiliza las respuestas de la IA cuando el usuario realiza preguntas relacionadas a la autoría. Optimiza de manera drástica el rendimiento de las uniones y subconsultas presentes en la *vista_catalogo_libros* y la *vista_autores_populares*.

Políticas de Claves Foráneas (FK)

- **Política de eliminación en cascada (ON DELETE CASCADE)**

Se optó por esta política en la relación entre la tabla *libro* y la tabla *ejemplar*. Un ejemplar no tiene sentido de existencia independiente si el libro al que pertenece (identificado por su isbn) desaparece por completo. En el caso de que se elimine un registro de la tabla *libro*, el motor borra de forma automática y en cadena todos sus ejemplares físicos asociados en la tabla *ejemplar*, evitando que queden registros huérfanos con claves que ya no apunten a ningún lado y de esa forma mantener la integridad de los datos.

- **Política de restricción por defecto (RESTRICT/NO ACTION)**

Para el resto de las relaciones del sistema donde no se especificó una cláusula explícita, MySQL aplica de forma automática la política de restricción. Esto ocurre en relaciones que resultan críticas como:

- *socio* referenciando a *tipoSocio* y *nacionalidad*
- *libro* referenciando a *editorial*
- *sancion* hacia *socio*, *descripcion* y *tipoSancion*
- *prestamo* hacia *socio*, *ejemplar* y *estadoPrestamo*

No se puede permitir la eliminación de un *socio* si este tiene registros vinculados en la tabla *prestamo* o *sancion*. Si se intenta borrar un socio con actividad, el motor arrojará un error bloqueando la acción. De esta manera se resguarda la integridad referencial de los datos.

Stored procedures y triggers

Stored procedures

sp_registrar_prestamo

Propósito: Registrar un nuevo préstamo verificando las reglas de negocio.

Parámetros:

- p_id_socio: identificador único del socio que pide el préstamo
- p_id_ejemplar: identificador del ejemplar físico del libro que se retira

Casos que maneja:

- Verifica que el socio no tenga sanciones activas. Si el socio está penalizado, deshabilitado o no existe, interrumpe la ejecución arrojando un error (*signal sqlstate '45000'*)
- Verifica que el socio no supere la cantidad de 3 préstamos activos en simultáneo. Si ya cuenta con el límite de préstamos activos en simultáneo establecidos como límite, se bloquea la transacción.
- Verifica la existencia y disponibilidad del ejemplar. Si no hay existencia del ejemplar en el stock de la biblioteca, el procedimiento falla evitando un stock negativo.
- Inserta el préstamo en caso de pasar las verificaciones de inserción correspondientes.

sp_registrar_devolucion

Propósito: Registrar la devolución de un préstamo y gestionar mora si corresponde.

Parámetros:

- p_id_prestamo: identificador único del registro de préstamo que se desea terminar

Casos que maneja:

- Verifica la existencia de un préstamo con ese identificador y que el mismo aún no se haya devuelto. Si no encuentra coincidencia, cancela la acción indicando que el préstamo ya se ha devuelto o no es válido.
- Asienta la devolución realizando un *update* fijando la fecha de devolución.
- Controla la devolución tardía, en caso de haber sido devuelto luego de la fecha de vencimiento, calcula la cantidad de días de atraso mediante la función *datedif()* e invoca de forma interna y automática al procedimiento *sp_generar_sancion*

sp_generar_sancion

Propósito: Crea una sanción proporcional a los días de mora.

Parámetros:

- p_id_socio: identificador único del socio penalizado
- p_tipo: tipo de sanción aplicada
- p_dias_mora: cantidad de días de retraso calculados previamente

Casos que maneja:

- Multiplica los días de mora por dos para establecer de forma interna la duración de la suspensión

- Inserta la fila en la tabla *sanción* fijando el inicio en la fecha actual y sumando el intervalo de días calculados.
- Inhabilita al socio dejándolo como inactivo para evitar que pueda realizar movimientos

sp_renovar_prestamo

Propósito: Extender la fecha de vencimiento de un préstamo activo.

Parámetros:

- p_id_prestamo: identificador único del préstamo activo que se pretende prorrogar.

Casos que maneja:

- Se comprueba que el préstamo exista y siga activo. En caso de que se haya devuelto o el ID es erróneo, se irrumpe el flujo.
- Rastrea si el socio está dado de baja o si tiene alguna sanción activa en el momento del pedido de renovación. En cualquiera de los casos, se le deniega la extensión del plazo.
- En el caso de éxito modifica el préstamo existente, aplazando la fecha de vencimiento por 14 días adicionales contados a partir del día de hoy.

sp_reactivar_socio

Propósito: habilitar un socio que ha sido dado de baja debido a una sanción.

Parámetros:

- p_id_socio: identificador único del socio.

Casos que maneja:

- Se verifica que el socio no tenga sanciones vigentes. Si tiene, no se hace nada.
- En caso de haber cumplido todas las sanciones, se lo vuelve a marcar como activo.

Triggers

trg_prestamos_simultaneos

Propósito: Actuar como una segunda barrera de seguridad a nivel de tabla para impedir la carga de préstamos excesivos de acuerdo a las reglas de negocio establecidas.

Casos que maneja: Evalúa la cantidad de filas sin devolver vinculadas al *id_socio* entrante. Si el conteo devuelve que ya tiene más de tres registros activos, se frena la operación antes de que se consolide el registro del nuevo préstamo.

trg_estado_socio

Propósito: Suspender automáticamente al socio cuando se genera una sanción

Casos que maneja: Actualiza el campo booleano *activo* del *socio* correspondiente a *false* (suspendido). Garantiza que ninguna ruta de inserción en la tabla *sancion* deje al socio en estado activo.

trg_actualizar_stock_insert

Propósito: Mantener actualizado en tiempo real el inventario disponible de libro cuando un ejemplar sale de la biblioteca.

Casos que maneja: Captura el *id* del ejemplar prestado, busca a qué libro pertenece en la tabla *ejemplar* y aplica un descuento matemático de una unidad en la tabla *libro*.

trg_actualizar_stock_update

Propósito: Restablecer el inventario disponible del catálogo de forma automática cuando un libro reingresa a la biblioteca.

Casos que maneja: Evalúa de forma estricta el cambio de estado de la fila modificada. Si la columna *fecha_devolucion* pasa de tener un valor vacío a un valor con fecha, se identifica el libro correspondiente y se incrementa en uno su *stock_disponible*.

trg_audit_prestamo_insert

Propósito: Iniciar la pista de auditoría histórica ante la creación de transacciones en el sistema.

Casos que maneja: Registra una nueva fila en *auditoria_prestamos*, guardando los datos del socio y ejemplar nuevos, dejando los valores anteriores en *null* e identificando el usuario de la base de datos que inició la acción mediante la función *USER()*.

trg_audit_prestamo_update

Propósito: Registrar cualquier modificación o actualización del estado de un préstamo para asegurar la trazabilidad técnica de los datos almacenados.

Casos que maneja: inserta una fila en *auditoria_prestamos*. Guarda de forma explícita el estado anterior y el estado nuevo de los campos neurálgicos, capturando los cambios de manera precisa.

trg_audit_prestamo_delete

Propósito: Prevenir y documentar la pérdida de información si algún registro de préstamo es borrado físicamente de la base de datos de manera accidental o malintencionada.

Casos que maneja: Se dispara de forma de almacenar la información y no perder información del registro eliminado. Almacena en la tabla de auditoría los datos históricos del registro a eliminar, dejando los campos nuevos en *null*.

Arquitectura del agente IA

El asistente de BibliolA opera bajo un sistema Text-to-SQL, traduciendo de forma directa las preguntas del usuario en lenguaje natural a consultas ejecutables de MySQL. El sistema funciona de manera independiente en cada iteración, esto significa que no almacena memoria de la conversación, sino que procesa cada consulta desde cero, adjuntando el esquema de la base de datos en cada llamada.

Flujo del proceso Text-to-SQL

1	El usuario escribe una pregunta en español, en lenguaje natural, dentro de la interfaz interactiva del cuaderno en Visual Studio Code.
2	Python construye el prompt del sistema combinando: la descripción del esquema de la base de datos, reglas de generación de código, ejemplos de preguntas y respuestas.
3	El prompt se envía a la API de Groq para que lo procese el modelo de inteligencia artificial, Llama-3.3-70b-versatile.
4	El modelo de IA analiza la pregunta y devuelve únicamente la consulta SQL correspondiente, sin agregar texto ni explicaciones adicionales.
5	Python recibe esa consulta generada y la ejecuta directamente sobre el motor de MySQL usando su conector nativo, recuperando las filas con los datos solicitados.
6	El resultado se convierte en un DataFrame de Pandas y se muestra formateado en la pantalla del Notebook de forma ordenada y fácil de leer para el usuario.

Prompt de Sistema

Al establecer que el modelo es un "traductor estricto", se anula su tendencia natural a conversar, justificar sus respuestas o agregar texto introductorio. Esta técnica de role prompting acota el espacio de comportamientos posibles del modelo.

Se definieron tres escenarios de falla controlada, cada uno con un mensaje de error predeterminado y literal. Esta técnica evita dos fallos típicos de los agentes Text-to-SQL: que el modelo "alucine" una tabla o columna inexistente para poder responder de todas formas, o que genere SQL sintácticamente válido pero semánticamente sin sentido.

Se listaron exhaustivamente las tablas y columnas correspondientes a cada tabla disponible, junto a las instrucciones reiteradas de no inventar columnas/alias/vistas/tablas dejando en claro las restricciones del esquema y para seguir previniendo las alucinaciones.

Los tres ejemplos seleccionados enseñan al modelo funciones esenciales y cómo utilizarlas de forma eficiente y útil:

- Ejemplo con *limit*: enseña al agente que, ante peticiones de listados, debe restringir el conjunto de datos devueltos. Se evitan lecturas masivas innecesarias en el disco
- Ejemplo con *like*: instruye al modelo sobre cómo interactuar eficientemente con los datos de texto (como buscar coincidencias parciales dentro de la cadena concatenada de géneros), simplificando la lógica de filtrado.

- Ejemplo con cruzamiento entre vistas: le demuestra al agente que, ante preguntas complejas como popularidad y categorías, no debe recurrir a las 17 tablas del esquema base ni intentar realizar uniones masivas (joins masivos). En su lugar, le enseña la técnica de fusionar vistas mediante la clave común del *isbn*, y de esta forma optimizar y acelerar el tiempo de respuesta.

Prompt

CONFIGURACIÓN DEL ASISTENTE BIBLIOIA (Text-to-SQL)

Sos un traductor estricto de lenguaje natural a MySQL 8.4.

REGLAS DE SALIDA

- * Tu respuesta debe contener únicamente una consulta SQL válida.
- * No utilizar markdown.
- * No utilizar bloques ```sql.
- * No escribir comentarios SQL.
- * No escribir explicaciones.
- * No escribir texto adicional.
- * La respuesta debe comenzar directamente con SELECT.

Si la pregunta no está relacionada con la biblioteca devolver exactamente:

```
SELECT 'Lo siento, no puedo ayudarte con eso' AS mensaje_sistema;
```

Si la consulta no puede resolverse utilizando exclusivamente las vistas o tablas listadas en este documento devolver exactamente:

```
SELECT 'Consulta no soportada por el esquema actual' AS mensaje_sistema;
```

Si alguna columna solicitada no existe devolver exactamente:

```
SELECT 'Columna inexistente en el esquema' AS mensaje_sistema;
```

TABLAS DISPONIBLES

Las únicas tablas existentes son las siguientes.

autor

Columnas:

- * id_autor
- * nombre
- * apellido

descripcion

Columnas:

- * id_descripcion
- * nombre

editorial

Columnas:

- * id_editorial
- * nombre

ejemplar

Columnas:

- * id_ejemplar
- * nro_ejemplar
- * id_estadoFisico
- * isbn

```
### estadoFisico
Columnas:
* id_estadoFisico
* nombre

### estadoPrestamo
Columnas:
* id_estadoPrestamo
* nombre

### genero
Columnas:
* id_genero
* nombre

### generoLibro
Columnas:
* id_genero
* isbn

### libro
Columnas:
* isbn
* titulo
* anio_publicacion
* stock_total
* stock_disponible
* edicion
* id_editorial

### libroAutor
Columnas:
* isbn
* id_autor

### nacionalidad
Columnas:
* id_nacionalidad
* nombre

### prestamo
Columnas:
* id_prestamo
* fecha_prestamo
* fecha_vencimiento
* fecha_devolucion
* id_socio
* id_ejemplar
* id_estadoPrestamo

### sancion
Columnas:
* id_sancion
* fecha_inicio
* fecha_fin
* id_descripcion
* id_tipoSancion
* id_socio
```

```
### socio
Columnas:
* id_socio
* dni
* nombre
* apellido
* email
* fecha_alta
* activo
* id_nacionalidad
* id_tipoSocio
```

```
### tipoSancion
Columnas:
* id_tipoSancion
* nombre
```

```
### tipoSocio
Columnas:
* id_tipoSocio
* nombre
```

Está prohibido utilizar cualquier tabla o columna que no aparezca explícitamente en esta lista.

```
## REGLAS DE GENERACIÓN
* Utilizar JOIN únicamente cuando sea necesario.
* Utilizar SIEMPRE alias distinguibles para referirse a tablas.
* No inventar columnas.
* No inventar alias semánticos.
* No inventar vistas.
* No inventar tablas.
* Verificar que cada columna exista antes de generar la consulta.
```

EJEMPLOS

```
Pregunta:
lista de 10 socios
Respuesta:
SELECT * FROM vista_socios_activos LIMIT 10;
```

```
Pregunta:
5 libros más prestados
Respuesta:
SELECT isbn, titulo, autores, total_prestamos
FROM vista_libros_populares
ORDER BY total_prestamos DESC
LIMIT 5;
```

```
Pregunta:
libros de economía más prestados
Respuesta:
SELECT c.isbn, c.titulo, c.autores, p.total_prestamos
FROM vista_libros_populares p
JOIN vista_catalogo_libros c
ON p.isbn = c.isbn
WHERE c.generos LIKE '%economia%'
ORDER BY p.total_prestamos DESC;
```

Evaluación del agente

Preguntas de prueba

=====

Preguntas mínimas

=====

Pregunta del usuario:

> ¿Cuáles son los 5 libros más prestados este año?

SQL GENERADO:

```
SELECT l.isbn, l.titulo, COUNT(p.id_prestamo) AS total_prestamos
FROM libro l
JOIN ejemplar e ON l.isbn = e.isbn
JOIN prestamo p ON e.id_ejemplar = p.id_ejemplar
WHERE YEAR(p.fecha_prestamo) = YEAR(CURDATE())
GROUP BY l.isbn, l.titulo
ORDER BY total_prestamos DESC
LIMIT 5;
```

Conectando a la base de datos...

Resultado:

	isbn	titulo	total_prestamos
0	978-11-99367-49-3	Semántica y pragmática del lenguaje natural	9
1	978-69-67717-33-9	Infecciones virales: epidemiología y control	9
2	978-39-60747-13-4	Antropología cultural: rituales y tradiciones	8
3	978-37-54328-83-4	Derecho internacional público y privado	7
4	978-10-01547-86-8	La violencia política en Occidente contemporáneo	6

Pregunta del usuario:

> ¿Qué socios tienen préstamos vencidos en este momento?

SQL GENERADO:

```
SELECT s.id_socio, s.nombre, s.apellido
FROM socio s
JOIN prestamo p
ON s.id_socio = p.id_socio
WHERE p.fecha_vencimiento < CURDATE() AND p.fecha_devolucion IS NULL;
```

Conectando a la base de datos...

Resultado:

	apellido	id_socio	nombre
0	Islami	285	Habibullah
1	Čubro	914	Tsimafei
2	Duncan	393	Naser
3	Al Shamsi	649	Zacharie
4	Mishra	639	Brayan
...	...	...	...
938	Safadi	67	Naledi
939	Cummings	19	Byiringiro
940	Di Pietro	160	Khamis
941	Kam	157	Grace
942	Claverol	62	Xiaojuan

943 rows × 3 columns

Pregunta del usuario:

> ¿Cuántos ejemplares disponibles hay del libro con ISBN 978-01-43433-88-8?

SQL GENERADO:

```
SELECT COUNT(e.id_ejemplar)
FROM ejemplar e
WHERE e.isbn = '978-01-43433-88-8' AND e.id_estadoFisico = 1
```

Conectando a la base de datos...

Resultado:

COUNT(e.id_ejemplar)
0
2

Pregunta del usuario:
> ¿Qué libros de Derecho están disponibles para prestar?

SQL GENERADO:
 SELECT l.isbn, l.titulo
 FROM libro l
 JOIN generoLibro gl ON l.isbn = gl.isbn
 JOIN genero g ON gl.id_genero = g.id_genero
 WHERE g.nombre = 'Derecho' AND l.stock_disponible > 0

Conectando a la base de datos...
 Resultado:

	isbn	titulo
0	978-00-43795-85-8	Ingeniería civil: cálculo estructural y resist...
1	978-03-30829-76-2	Algebra lineal avanzada: espacios vectoriales,...
2	978-04-84183-06-6	Hermenéutica textual: interpretación de símbol...
3	978-04-99217-72-5	Derecho de familia: matrimonio, filiación y su...
4	978-07-09631-11-2	Computación gráfica: modelado 3D y técnicas de...
5	978-11-61761-93-2	Desarrollo de aplicaciones móviles: arquitectu...
6	978-13-05516-52-5	Endocrinología: diabetes mellitus y trastornos...
7	978-14-01174-26-6	Lírica romántica española: melancolía, pasión ...
8	978-16-63222-12-1	Seguridad informática: criptografía, ataques y...
9	978-17-26678-69-9	Pediatría general: crecimiento y desarrollo de...
10	978-17-41132-66-8	Filosofía del lenguaje: referencia, intenciona...
11	978-22-21202-70-9	Filosofía política: contrato social, legitimid...
12	978-28-90001-97-4	Oncología pediátrica: leucemias, linfomas y tu...
13	978-29-06197-16-1	Novela negra contemporánea: femicidio y justic...
14	978-31-55853-12-1	Novela gótica latinoamericana: horror colonial...
15	978-37-54328-83-4	Derecho internacional público y privado
16	978-39-14700-69-8	Poesía de protesta: denuncia social y compromi...
17	978-42-35458-25-0	Derecho administrativo: función pública, proce...
18	978-43-81891-06-5	Astrofísica: formación de estrellas y agujeros...
19	978-47-32653-99-8	Infectología: HIV, tuberculosis multirresisten...
20	978-52-38305-30-0	Machine learning: clustering, regresión y vali...
21	978-54-26709-52-6	Urología: hiperplasia prostática y litiasis ur...
22	978-56-03113-20-4	Infectología: tuberculosis, VIH/SIDA y enferme...
23	978-63-69282-74-1	Bioquímica celular: metabolismo energético y s...
24	978-64-99128-59-5	Regulación de radiofármacos: medicina nuclear,...
25	978-71-73296-95-5	Algoritmos de aprendizaje automático: regresió...
26	978-75-39953-04-8	Literatura de ciencia ficción: futuros distópi...
27	978-85-24860-23-9	Breve historia del universo y sus misterios
28	978-86-85187-47-7	Psicología del desarrollo humano
29	978-92-19514-22-1	Constitucionalismo democrático: derechos funda...
30	978-94-24010-64-5	Neumología: asma, enfermedad pulmonar obstruct...
31	978-99-56011-21-8	Arquitectura de microservicios: escalabilidad ...

Pregunta del usuario:

> ¿Cuál es el historial de préstamos del socio con DNI 47910299?

SQL GENERADO:

```
SELECT p.fecha_prestamo, p.fecha_vencimiento, p.fecha_devolucion, e.nro_ejemplar, l.titulo
FROM prestamo p
JOIN ejemplar e ON p.id_ejemplar = e.id_ejemplar
JOIN libro l ON e.isbn = l.isbn
JOIN socio s ON p.id_socio = s.id_socio
WHERE s.dni = 47910299
```

Conectando a la base de datos...

Resultado:

	fecha_devolucion	fecha_prestamo	fecha_vencimiento	nro_ejemplar	titulo
0	2024-12-08	2024-11-22	2024-12-06	1	Farmacología clínica y terapéutica
1	None	2022-05-30	2022-06-20	1	Filosofía de la mente: conciencia, materialism...
2	2023-03-10	2023-03-08	2023-03-29	2	Flebología: insuficiencia venosa, trombosis y ...
3	None	2022-08-16	2022-09-06	1	Redes neuronales artificiales: aprendizaje pro...

Pregunta del usuario:

> ¿Qué autores tienen más de 3 libros en la biblioteca?

SQL GENERADO:

```
SELECT a.nombre, a.apellido, COUNT(DISTINCT la.isbn) AS cantidad_libros
FROM autor a
JOIN libroAutor la
ON a.id_autor = la.id_autor
GROUP BY a.id_autor, a.nombre, a.apellido
HAVING COUNT(DISTINCT la.isbn) > 3;
```

Conectando a la base de datos...

Resultado:

	apellido	cantidad_libros	nombre
0	Lai	5	Matthew
1	Barrientos	4	Andrea
2	Macia	5	Martin
3	Hyppolite	4	Alain
4	Volkov	5	Ignace
...	...	...	...
80	Celestin	8	Moosa
81	Khamis	4	Josef
82	Naji	6	Catherine
83	Barthel	4	'Alisi
84	Brajović	7	Bangura

85 rows × 3 columns

Pregunta del usuario:
> ¿Cuántas sanciones se generaron en el último mes?

SQL GENERADO:
SELECT COUNT(*)
FROM sancion s
WHERE s.fecha_inicio >= NOW() - INTERVAL 1 MONTH

Conectando a la base de datos...
Resultado:

COUNT(*)
14

Pregunta del usuario:
> ¿Qué socios tienen más de 3 préstamos históricos?

SQL GENERADO:
SELECT s.id_socio, s.nombre, s.apellido, COUNT(p.id_prestamo) AS total_prestamos
FROM socio s
JOIN prestamo p ON s.id_socio = p.id_socio
GROUP BY s.id_socio, s.nombre, s.apellido
HAVING COUNT(p.id_prestamo) > 3;

Conectando a la base de datos...
Resultado:

	apellido	id_socio	nombre	total_prestamos
0	Abbott	557	Hawa	5
1	Abdrakhmanov	147	Tehani	5
2	Adkins	775	Kahsay	5
3	Ahmed	617	Tomáš	6
4	Akel	611	Ogola	5
...	...	...	...	...
379	Yassin	967	Sarai	4
380	Young	486	Luz	4
381	Yousef	22	Sering	5
382	Zamore	161	Savvas	5
383	Zidi	988	Madison	4

384 rows × 4 columns

```
Pregunta del usuario:
> cuales son los 3 libros mas prestados de medicina

SQL GENERADO:
SELECT l.isbn, l.titulo, COUNT(p.id_prestamo) AS total_prestamos
FROM libro l
JOIN ejemplar e ON l.isbn = e.isbn
JOIN prestamo p ON e.id_ejemplar = p.id_ejemplar
JOIN generoLibro gl ON l.isbn = gl.isbn
JOIN genero g ON gl.id_genero = g.id_genero
WHERE g.nombre = 'medicina'
GROUP BY l.isbn, l.titulo
ORDER BY total_prestamos DESC
LIMIT 3;

Conectando a la base de datos...
Resultado:
```

	isbn	titulo	total_prestamos
0	978-93-55953-01-8	Neumología: enfermedad pulmonar obstructiva cr...	16
1	978-98-97838-87-5	Historia del arte renacentista: humanismo, per...	14
2	978-96-50682-99-8	Endocrinología: diabetes mellitus, hipotiroidi...	13

```
Pregunta del usuario:
> cual es el libro con mas ejemplares

SQL GENERADO:
SELECT l.titulo, SUM(e.nro_ejemplar) AS total_ejemplares
FROM libro l
JOIN ejemplar e
ON l.isbn = e.isbn
GROUP BY l.titulo
ORDER BY total_ejemplares DESC
LIMIT 1;

Conectando a la base de datos...
Resultado:
```

	titulo	total_ejemplares
0	Arquitectura de software: patrones y refactori...	36

Se realizaron más de 10 pruebas con preguntas de distinta complejidad (consultas simples, agregación, filtros, ordenamientos y JOINS). En todos los casos el agente generó consultas SQL válidas y obtuvo resultados coherentes con la base de datos. No se detectaron errores de sintaxis SQL durante las pruebas realizadas. Las principales limitaciones observadas están relacionadas con consultas ambiguas o preguntas que requieren información no disponible en el esquema de datos.

Módulo de recomendaciones

El sistema incorpora un módulo de recomendaciones basado en IA cuyo objetivo es sugerir nuevas lecturas personalizadas a cada socio de la biblioteca a partir de su historial de préstamos.

El proceso comienza cuando el usuario ingresa un identificador de socio, que puede corresponder al número de socio o al DNI. A partir de este dato, el sistema consulta la base de datos para recuperar los autores y géneros de los libros que el socio ha leído previamente. Esta información se utiliza para construir un perfil básico de preferencias literarias.

Posteriormente, el sistema obtiene una lista de libros candidatos disponibles en la biblioteca. Para evitar recomendaciones redundantes, se excluyen aquellos títulos que ya fueron prestados al socio. Además, únicamente se consideran libros con stock disponible para préstamo.

Una vez obtenida la información necesaria, se construye un prompt que incluye:

- Los autores previamente leídos por el socio.
- Los géneros de interés identificados a partir de su historial.
- La lista de libros candidatos disponibles.
- Un conjunto de instrucciones que restringen el comportamiento del modelo.

El prompt es enviado al modelo de lenguaje ejecutado mediante la API de Groq. El modelo recibe instrucciones explícitas para seleccionar exactamente tres libros de la lista proporcionada, priorizando coincidencias con los gustos previamente detectados.

Asimismo, se le exige responder exclusivamente en formato JSON para facilitar el procesamiento automático de la respuesta.

La respuesta generada por el modelo es validada y convertida a un DataFrame de Pandas, que posteriormente es mostrado al usuario. Cada recomendación incluye el título del libro, el autor, el género y una breve justificación generada por la IA explicando porque dicho libro resulta adecuado para el socio.

El módulo también contempla el manejo de errores y casos especiales. Si el socio no posee historial suficiente para inferir preferencias, o si no existen libros disponibles para recomendar, el sistema informa la situación mediante mensajes descriptivos. Del mismo modo, se validan posibles errores de formato en la respuesta generada por la IA.

Ejemplos de salida

```
Pregunta del usuario:
> recomendar 3

Generando recomendaciones...

  titulo  autor  genero  motivo
0               El socio no posee historial suficiente para ge...
```

Pregunta del usuario:

> recomendar 1

Generando recomendaciones...

Cantidad filas: 1

autor_nombre genero_nombre

0 Intissar Gauci Religión

campo_busqueda = id_socio

dni_o_id = 1

```
SELECT DISTINCT
  CONCAT(a.nombre, ' ', a.apellido) AS autor_nombre,
  g.nombre AS genero_nombre
FROM socio s
JOIN prestamo p ON s.id_socio = p.id_socio
JOIN ejemplar e ON p.id_ejemplar = e.id_ejemplar
JOIN libro l ON e.isbn = l.isbn
JOIN libroAutor la ON l.isbn = la.isbn
JOIN autor a ON la.id_autor = a.id_autor
JOIN generoLibro gl ON l.isbn = gl.isbn
JOIN genero g ON gl.id_genero = g.id_genero
WHERE s.id_socio = :valor
```

	titulo	autor	genero	motivo
0	Crítica de la razón práctica	Rebecca Beda	Religión	Coincide con el género preferido del socio (Re...
1	Procesamiento de lenguaje natural: tokenizació...	Rebecca Beda	Tecnología	Coincide con un autor que tiene un libro en el...
2	Crítica de la razón práctica	Rebecca Beda	Teatro	No hay coincidencias con el autor preferido, p...

Pregunta del usuario:

> recomendar 100

Generando recomendaciones...

Cantidad filas: 1

autor_nombre genero_nombre

0 Agnes Vesiloo Psicología

campo_busqueda = id_socio

dni_o_id = 100

```
SELECT DISTINCT
  CONCAT(a.nombre, ' ', a.apellido) AS autor_nombre,
  g.nombre AS genero_nombre
FROM socio s
JOIN prestamo p ON s.id_socio = p.id_socio
JOIN ejemplar e ON p.id_ejemplar = e.id_ejemplar
JOIN libro l ON e.isbn = l.isbn
JOIN libroAutor la ON l.isbn = la.isbn
JOIN autor a ON la.id_autor = a.id_autor
JOIN generoLibro gl ON l.isbn = gl.isbn
JOIN genero g ON gl.id_genero = g.id_genero
WHERE s.id_socio = :valor
```

	titulo	autor	genero	motivo
0	Teoría de la relatividad general: gravitación ...	Joanne Alefaio	Psicología	Coincide con el género preferido del socio (Ps...
1	Pragmática del lenguaje: actos de habla y cont...	Rui Azopardi	Psicología	Coincide con el género preferido del socio (Ps...
2	Neumología: asma, enfermedad pulmonar obstruct...	Joanne Alefaio	Psicología	Coincide con el género preferido del socio (Ps...

Conclusiones

Lecciones aprendidas

Notamos que pasarle las tablas crudas a la IA resultaba caótico debido que se confundía con los joins, devolvía cosas sin sentido. La lección en este punto fue descubrir que al crear vistas se le aligeraba el trabajo a la IA de forma que podía lograr consultas más simples y resultados exitosos.

Observamos la ventaja de utilizar triggers y stored procedures, al dejar el control de los límites de préstamos o la baja automática de socios dentro de MySQL nos aseguramos que aunque la IA alucine, el motor no permite que se rompan las reglas de negocios.

Limitaciones del agente

En lo que respecta a las limitaciones del agente, pudimos observar que se vuelve sensible frente a ambigüedades idiomáticas o estructuras lingüísticas informales que utilicen términos no declarados de forma explícita en el diccionario del prompt. Si la instrucción no es completamente clara, el modelo pierde precisión y puede confundir los nombres de las columnas, o incluso "alucinar" inventando tablas, campos o vistas que no existen en la base de datos real, lo que hace que la consulta falle al ejecutarse en MySQL.

El agente procesa cada solicitud de manera aislada, al no contar con un manejo de contexto histórico (no tiene memoria conversacional), no logra resolver de forma secuencial preguntas que dependen directamente de la respuesta anterior (por ejemplo: "¿cuáles son los socios morosos?" y seguido: ¿cuánto debe el primero?).

Mejoras posibles

En nuestro proyecto, si agregamos una columna en MySQL, tenemos que ir a modificar el prompt a mano. Una mejora real sería lograr que se lea automáticamente el esquema y se actualice ese texto de forma dinámica antes de llamar a la IA.

Para el módulo de recomendaciones, se podrían tener en cuenta muchos atributos más, como el tipo y la edad del socio.

Bibliografía y recursos utilizados

Datos de ejemplo

- Tonic Fabricate

Herramientas utilizadas

- Python 3.x
- MySQL 8.4
- SQLAlchemy
- Pandas
- Jupyter Notebook
- Docker y Docker Compose
- Git y GitHub
- Visual Studio Code
- API de Groq para inferencia de modelos LLM

Recursos de IA utilizados

- ChatGPT (OpenAI)
- Gemini (Google)
- Claude (Anthropic)